

## LECTURE 29

# Storage Device Management

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

# In This Lecture

- Low-level (physical) formatting vs. logical formatting, and what each one actually writes to the disk
- The boot block and the full bootstrap chain, from power-on to a running OS
- How disks handle bad blocks — and why this is almost invisible to the OS on modern hardware
- Swap-space management: what swap space is for, where it can live, and why it is allocated differently from ordinary file storage

# Formatting a Disk

A disk has to be formatted twice, at two different levels, before it is useful — and the two are easy to confuse because both are casually called “formatting.”

Two formatting steps, two different jobs

## Low-level (physical) formatting

Divides the disk into addressable sectors with ECC — done at the factory, rarely by end users

## Logical formatting

Writes a boot block, free-space info, and an empty root directory — what “formatting a drive” usually means today

# Low-Level (Physical) Formatting

Divides a raw disk into sectors the disk controller can read and write, writing a header and a trailer around each sector's data area.

- The header and trailer record a sector number and a piece of error-correcting code (ECC)
- The controller later uses the ECC to detect — and often repair — corrupted data automatically whenever that sector is read back
- Decades ago, buying a new hard disk meant running a low-level format program before it could be used at all
- Today, manufacturers perform low-level formatting at the factory, as part of manufacturing and testing
- End users and admins only encounter it through specialized diagnostic or recovery tools, and only when a disk is already behaving abnormally

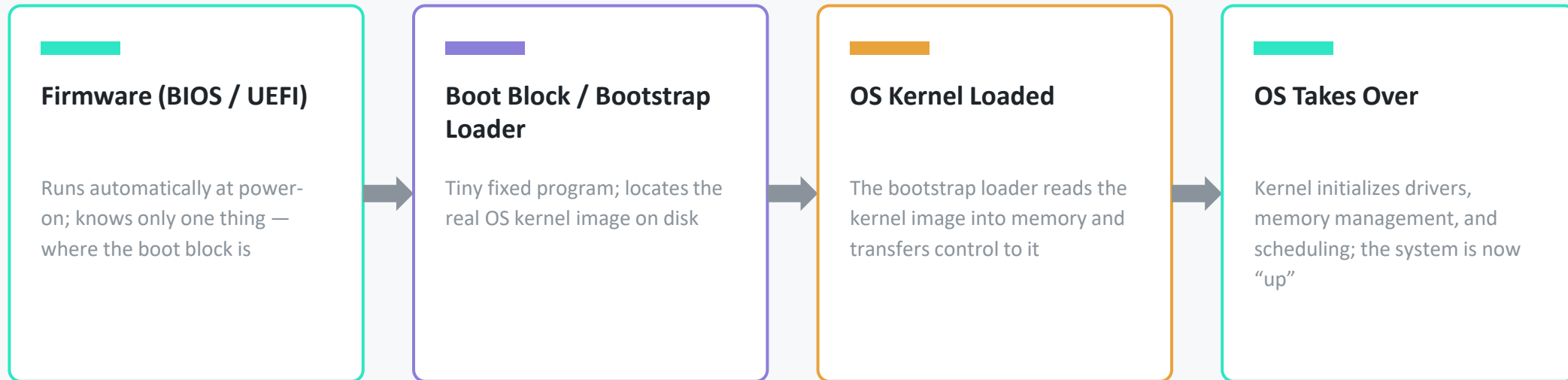
# Logical Formatting

The step an operating system performs to actually make a disk usable as a volume: it writes the initial file-system data structures onto the already physically formatted disk.

- A boot block, reserved at a fixed, known location so firmware can always find it
- Free-space information, recording which blocks on the volume are available for new data (a bitmap or a free list)
- An initial, empty root directory, the one directory every other file and directory on the volume is eventually reachable from
- This is the step most people actually mean when they say “format this drive” — reformatting a USB stick from FAT32 to NTFS is a logical format, not a physical one, and is dramatically faster because the physical sector boundaries underneath are left untouched

# The Boot Block and the Bootstrap Process

The bootstrap loader is a small program whose only job is to locate the real OS kernel, load it into memory, and jump to its starting address — stored in a reserved, fixed area called the boot block.



# Why Each Link Stays Simple

- Firmware is burned into read-only (or flash) memory on the motherboard, so it survives even a completely empty or corrupted disk — its only job is a power-on self-test, then reading one small, fixed region of the boot device
- The boot block/bootloader is small enough to fit in that fixed region and simple enough not to need a working file system — it finds the kernel by its known location, not by searching a directory tree
- The OS kernel, once loaded, is finally a fully capable program — it can initialize device drivers, set up virtual memory, and start the scheduler, none of which the tiny bootstrap loader could ever do

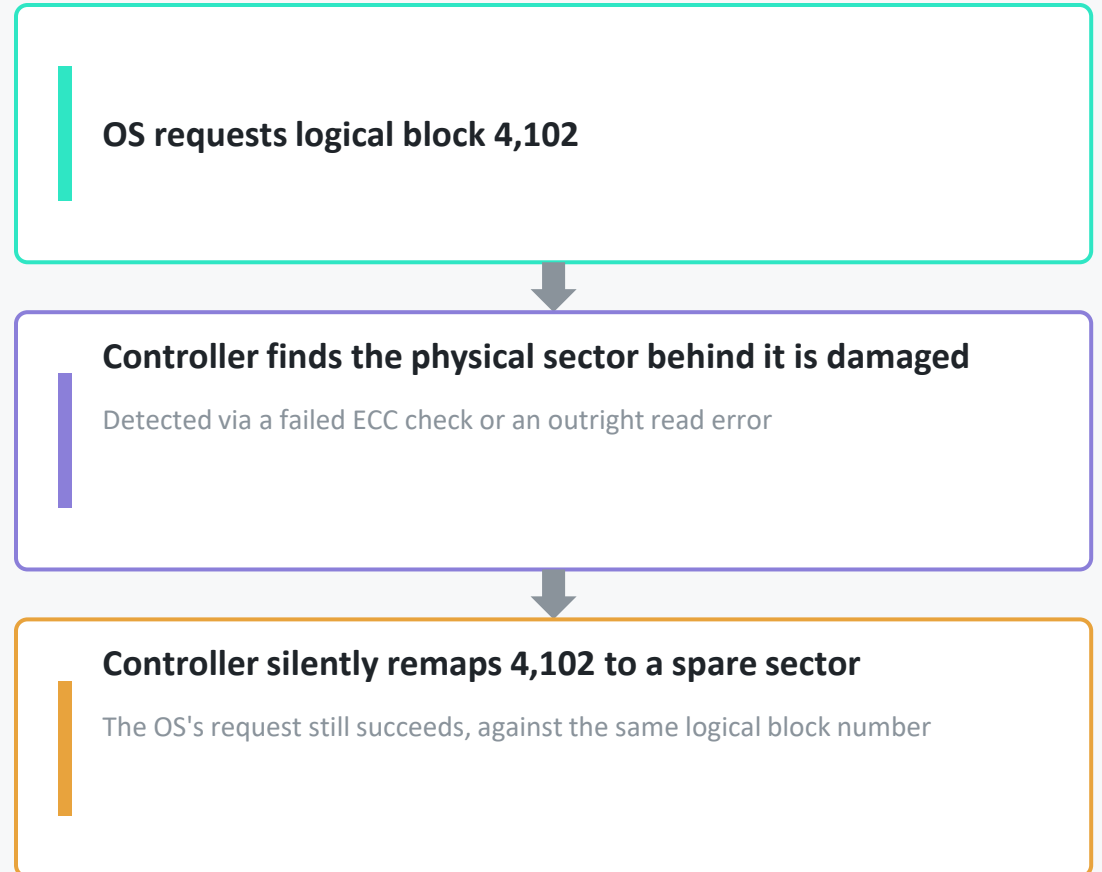
## Why not have the firmware load the OS directly?

Firmware is written once, flashed onto the motherboard, and rarely updated — it has to work with any operating system a user might install, from any vendor, in any file-system format. Splitting the job lets the firmware stay completely generic (“read this one fixed block and run whatever is there”) while the bootloader, which is specific to one OS and file system, handles everything beyond that. Change operating systems, and only the boot block needs to change — the firmware never does.

# Bad Blocks

Disks eventually develop physically damaged sectors — a scratch on a platter, or a flash cell worn out after too many writes. Modern disks handle this almost entirely in hardware: the disk controller maintains a reserved spare-sector pool set aside specifically for this purpose.

From the operating system's point of view, nothing changed — the same logical block number still works, and the OS never needs to know a remap even happened.



# Bad Blocks: Before Hardware Remapping

- Older, simpler disk systems — and some low-level utilities even today — had no spare-sector pool at all
- Bad blocks were tracked in software instead: a dedicated bad-block list, maintained by the OS or a formatting utility
- The file system had to be told explicitly to avoid allocating any file data to those specific blocks
- A sector going bad on a disk like this could mean real data loss, since nothing beneath the OS was watching out for it

## Spare sectors are a mitigation, not a guarantee

Hardware remapping handles sectors failing one at a time, gracefully, as a disk ages. It does not protect against the drive failing wholesale, nor does a nearly-exhausted spare-sector pool continue working forever — a disk reporting a rapidly growing reallocated-sector count is a strong signal it is approaching end of life, regardless of how invisible each individual remap has been so far.

# Swap-Space Management

Swap space is a reserved area of secondary storage the OS uses to hold pages (and, on some systems, entire processes) that do not currently fit in physical memory — what makes usable memory larger than physical RAM, at the cost of disk-speed access on a miss.

## Where swap space can live

### A raw disk partition

Managed directly by the OS's own swap-space storage manager, bypassing the file system entirely — faster, since there is no file-system overhead on every swap I/O

### A regular file inside a file system

A normal file (e.g. a Windows pagefile) the OS treats as swap space — more flexible: it can be resized, moved, or removed using ordinary file operations

# Swap I/O Performance Matters

- A page fault that has to wait on a slow swap read stalls the process — and often other processes scheduled behind it — far more visibly than almost any other kind of disk access
- Swap space is typically allocated in large, fixed-size chunks, rather than left to fragment across the disk the way ordinary file storage is allowed to
- Large, contiguous, uniformly sized swap extents keep the access pattern simple and predictable for the swap manager, which can then address a swapped-out page with simple arithmetic instead of walking a file-system's block-allocation structures on every page-in and page-out

## Swap space is a consequence of Lectures 25–26, not a separate topic

Everything here exists purely to support demand paging and page replacement: a page a process still needs, but that memory pressure forced out of RAM, has to go somewhere that can be read back in quickly on the next page fault. Swap space is that somewhere — storage management and memory management meet directly at this one point.

# Key Takeaways

- Low-level (physical) formatting divides a disk into addressable sectors with ECC, done at the factory; logical formatting writes a boot block, free-space information, and an empty root directory — what most people mean by “formatting a drive” today
- The bootstrap chain keeps each stage as simple as possible: firmware reads one fixed location (the boot block), the bootstrap loader locates and loads the real OS kernel, and only then does the kernel take over
- Modern disks mostly hide bad blocks from the OS, transparently remapping damaged logical blocks onto a reserved spare-sector pool; older systems had to track bad blocks explicitly in software
- Swap space extends usable memory beyond physical RAM by holding evicted pages, can live on a raw partition (faster) or inside a regular file (more flexible), and is allocated in large fixed-size chunks because swap I/O performance is critical to system responsiveness